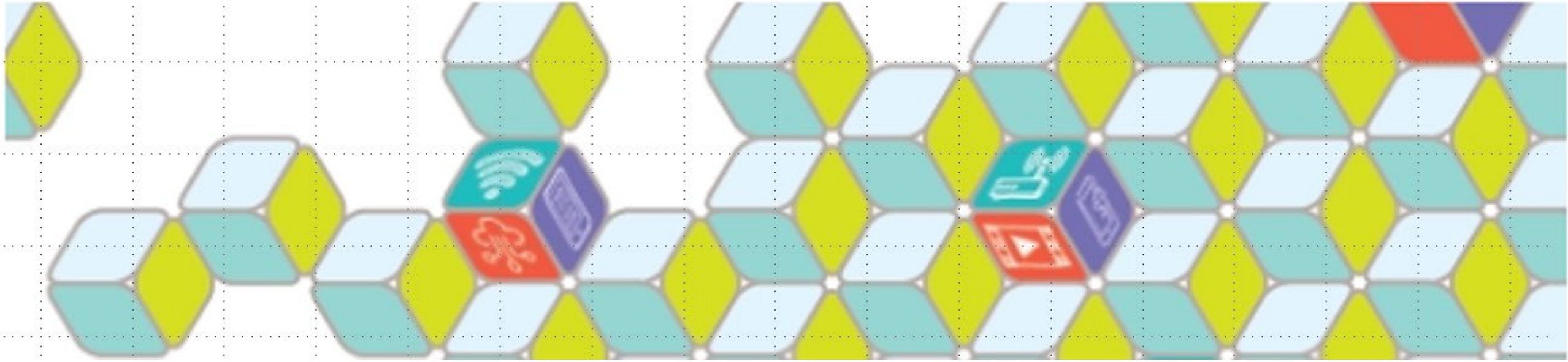


Programming Method
(using C, C++, for Games)

Prof. Tae Soo Yun
Dept. of Game, International College, Dongseo University

DSU Dongseo University 동서대학교

Ch03_Control Flow

The Goal of the ch. 03-Control Flow

The goal of ch03 : understanding the Control flow statements of a language specify the order in which computations are done.

Chapter 3

- Section 3.1 - Statements and Blocks
- Section 3.2 If-Else
- Section 3.3 Else-If
- Section 3.4 Switch - Never use ??
- Section 3.5 Loops - While and For
- Section 3.6 Loops - Do-While
- Section 3.7 Breaks
- Section 3.8 Continue
- Section 3.9 Goto's and Labels

3.1 Statements and Block

❖ An expression (x=0 or i++ or printf(…) becomes a statement is followed by a semicolon, as in ☞

```
x =0;
i++;
Printf(…);
```

✓ Semicolon is a statement terminator not a separator

❖ Block, Compound statement

✓ braces { and } are used to group declarations and statements together into a compound statement or block ☞

✓ Are syntactically equivalent to a single statement.

- Braces surround the statement of a function
- Braces surround multiple statements after an *if, else, while or for* etc., be careful to use { }

```
#include <stdio.h>
int main() {
    char x[10];
    int i;

    for(i=0; i<1000; i++ ) x[i] = '*';
    printf("%s\n",x);
}
```

```
if(c == '\n') {
    s[i] = c;
    ++i;
}
```

```
if(c == '\n') {
    s[i] = c;
}
++i;
```

3.2 If-Else

❖ If-else statements is used to (express)make decisions.

✓ syntax

```
if(expression)
        statement-1
else
        statement-2
```

If expression has a non-zero value

✓ where the else part is optional. The expression is evaluated; if it is “true”, statement-1 is executed. If it is “false” and if there is an else part, statement-2 is executed instead.

✓ Ambiguity problem.

- else part is optional; ambiguity arises when an else is omitted from a nested if sequence.
- The "else" part is affected by the inner "if" statement.

```
if(n > 0)
     if(a > b)
         z = a;
     else
         z = b;
```

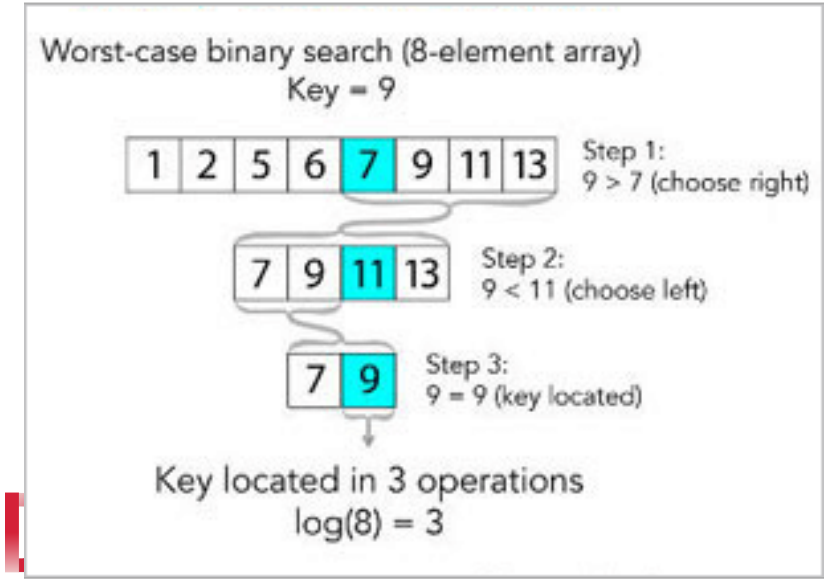
```
if(n > 0){
     if(a > b)
         z = a;
}
else
     z = b;
```

3.3 Else-if

❖ else-if statement used to write a multi-way decision.

```
if(expression)
      statement
else if (expression)
      statement
else if (expression)
      statement
[else
      statement]
```

Binary Search Diagram




```
[ex. 2]three-way decision,
binary search function that decide if a
particular value x occurs in array v
binary(x,v,n) /*find x in v[0] … v[n-1]
int x, v[], n;
{
      int low, high, mid;
      low = 0;
      high = n-1;
      while(low <= high){
          mid = (low+high) /2;
          if(x<v[mid])
             high = mid-1;
          else if(x>v[mid])
             low = mid + 1;
          else /*found match*/
      }
      return(-1);
}
```

3.4 Switch

- is a special multi-way decision that tests whether an expression matches one of a number of constant values, and branches accordingly.

```
Switch (expression){
    case const-expr: statements
    case const-expr: statements
    default: statements
}
```

- Is especially useful when the selection is based on the value of a single variable or a simple expression- may be int, char, but not double.

- [ex.3] The same example in chapter 01, a program to count the occurrences of each digit, white space, and all other characters.

[ex. 3]

```
#include <stdio.h>

main()  /* count digits, white space, others */
{
    int c, i, nwhite, nother, ndigit[10];

    nwhite = nother = 0;
    for (i = 0; i < 10; i++)
        ndigit[i] = 0;
    while ((c = getchar()) != EOF) {
        switch (c) {
        case '0': case '1': case '2': case '3': case '4':
        case '5': case '6': case '7': case '8': case '9':
            ndigit[c-'0']++;
            break;
        case ' ':
        case '\n':
        case '\t':
            nwhite++;
            break;
        default:
            nother++;
            break;
        }
    }
    printf("digits =");
    for (i = 0; i < 10; i++)
        printf(" %d", ndigit[i]);
    printf(", white space = %d, other = %d\n",
        nwhite, nother);
    return 0;
}
```

3.4 Switch-continue

- Break statement causes an immediate exit from the switch. Just as label.
- break also be used to exit from while, for, and do loops.
- Recommend put a break after the last case even though it's logically unnecessary.

[ex. 3]

```
#include <stdio.h>

main()  /* count digits, white space, others */
{
    int c, i, nwhite, nother, ndigit[10];

    nwhite = nother = 0;
    for (i = 0; i < 10; i++)
        ndigit[i] = 0;
    while ((c = getchar()) != EOF) {
        switch (c) {
        case '0': case '1': case '2': case '3': case '4':
        case '5': case '6': case '7': case '8': case '9':
            ndigit[c-'0']++;
            break;
        case ' ':
        case '\n':
        case '\t':
            nwhite++;
            break;
        default:
            nother++;
            break;
        }
    }
    printf("digits =");
    for (i = 0; i < 10; i++)
        printf(" %d", ndigit[i]);
    printf(", white space = %d, other = %d\n",
        nwhite, nother);
    return 0;
}
```

❖ Exercise 3-2

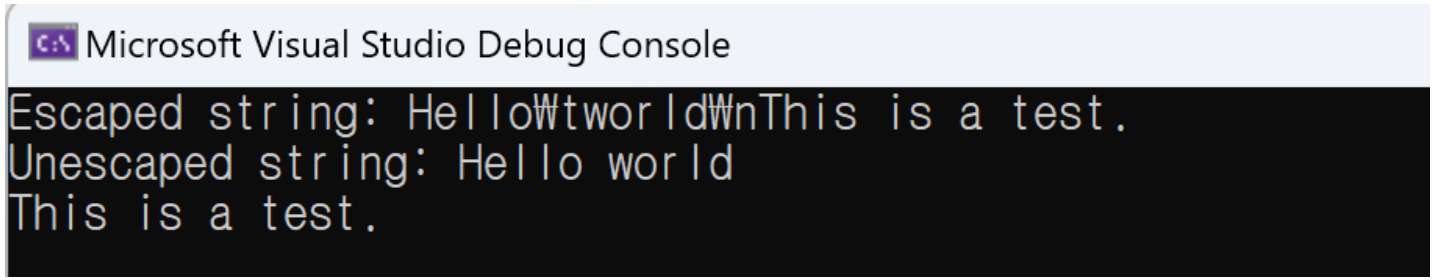
✓ Write a function escape(s,t) that converts characters like newline and tab into visible escape sequences like ₩n and ₩t as it copies the string t to s. Use a switch. Write a function for other direction as well, converting escape sequences into the real characters.

✓ Suppose that input t[] = "Hello₩tworld₩nThis is a test."

- write escape(s,t) that copies characters from t to s, but converts newline(₩n) and tab(₩t) characters into their respective escape sequences(₩₩n, ₩₩t)

- and unescaped(s,t) that convert escape sequences(₩₩n, ₩₩t) back into their corresponding special characters(₩n,₩t)

✓ Answer :



3.5 Loops-While and For

- in `while(expression)`
  `statement`

- the expression is evaluated. If it is non-zero, statement is executed and expression is re-evaluated. This cycle continues until exp. becomes zero.
- the for statement for(...) is equivalent to while (…), as shown in right hand side. ☞
- In for loop, expr1 and expr3 are assignments or function calls, expr2 is a relational expression.
- ie, expr1 is start condition, expr2 is end condition, expr3 is step direction (++, --)
- If expr1 or expr3 is omitted, it is simply dropped from the expansion.
- If the test, expr2, is not present, it is taken as permanently true, so `for( ; ; ) { … }` is an infinite loop,

```
for(expr1 ; expr2 ; expr3)
        statement
```

```
expr1;
while(expr2){
        statement
        expr3;
}
```

3.5 Loops-While and For (continue)

❖ when or what should we use while or for loop ?

- ✓ Is a matter of personal preference.
- ✓ No initialization or re-initialization, while is most natural.
- ✓ The for is preferable when there is an initialization and increment since it keeps the loop control statements close together.

```
for (i=0 ; i < n; i++) {
        …
}
```

3.6 Loops - do-while

- the syntax of the do-while is

```
do
    statement
while( expression );
```

- the statement is executed, the expression is evaluated.
- do-while is much less used than while, for. Nonetheless, from time to time it is valuable, as in the following itoa. (the inverse of atoi)
  - ✓ In itoa, the do-while is necessary, or at least convenient, since at least one character must be installed in the array s, even if n is zero.
  - ✓ itoa function generates the digit to string backward, and the reverse(s) reverse it.

```
/* itoa:  convert n to characters in s */
void itoa(int n, char s[])
{
    int i, sign;

    if ((sign = n) < 0)  /* record sign */
        n = -n;          /* make n positive */
    i = 0;
    do {       /* generate digits in reverse order */
        s[i++] = n % 10 + '0';   /* get next digit */
    } while ((n /= 10) > 0);     /* delete it */
    if (sign < 0)
        s[i++] = '-';
    s[i] = '\0';
    reverse(s);
}

#include <string.h>

/* reverse:  reverse string s in place */
void reverse(char s[])
{
    int c, i, j;

    for (i = 0, j = strlen(s)-1; i < j; i++, j--) {
        c = s[i];
        s[i] = s[j];
        s[j] = c;
    }
}
```

Exercise 3-4. In a two's complement number representation, our version of `itoa` does not handle the largest negative number, that is, the value of `n` equal to $-(2^{wordsize-1})$. Explain why not. Modify it to print that value correctly, regardless of the machine on which it runs. □

Exercise 3-5. Write the function `itob(n,s,b)` that converts the integer `n` into a base `b` character representation in the string `s`. In particular, `itob(n,s,16)` formats `n` as a hexadecimal integer in `s`. □

Exercise 3-6. Write a version of `itoa` that accepts three arguments instead of two. The third argument is a minimum field width; the converted number must be padded with blanks on the left if necessary to make it wide enough. □

3-7 Break and Continue

- Sometimes we need to exit from a loop.
- break statement provides an early exit from for,while, and do. Just as from switch.
  - ✓ a break causes the innermost enclosing loop or switch
- ex) the r.s.h example,trim, use a break to exit from a loop when the rightmost non-blank, not tab, non-newline is found.

```
/* trim:  remove trailing blanks, tabs, newlines */
int trim(char s[])
{
    int n;

    for (n = strlen(s)-1; n >= 0; n--)
        if (s[n] != ' ' && s[n] != '\t' && s[n] != '\n')
            break;
    s[n+1] = '\0';
    return n;
}
```

- Continue statement, it causes the next iteration of the enclosing for, while, or do loop to begin.
  - ✓ Continue statement applies only to loops, not to switch.

```
for (i = 0; i < n; i++) {
    if (a[i] < 0)    /* skip negative elements */
        continue;
    ...   /* do positive elements */
}
```

3.8 Goto and Labels

❖ Goto statement is used to abandon processing in some deeply nested structure, such as breaking out of two or ore loops at once.

- ✓ Break statement cannot be used directly since it only exits from the innermost loop

```
for ( ... )
    for ( ... ) {
        ...
        if (disaster)
            goto error;
    }
...

error:
    clean up the mess
```

❖ A label has the same form as a variable name, and is followed by a colon.

```
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        if (a[i] == b[j])
            goto found;
/* didn't find any common element */
...
found:
    /* got one:  a[i] == b[j] */
    ...
```